



Academic Year: 2025-26

Soundalgekar

Semester: 4

Class / Branch: SE COMPS A

Subject: OS

Name of Instructor: Prof. Shamika Mule

Name of Student: Shriya

Student ID: 24102054

Date of performance: 2/4/26

Date of submission: 2/4/26

Experiment No. 8

Aim: Write a program to implement page replacement policy LRU.

Program:

```
#include <stdio.h>
```

```
int input[20] = {7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};
```

```
int sol[3];
```

```
int status[3]; // last used time
```

```
// Check if page is present
```

```
int present(int n)
```

```
{
```

```
for(int i = 0; i < 3; i++)
```

```
{
```

```
if(sol[i] == n)
```

```
return i; // return index if found
```

```
}
```

```
return -1; // not found
```

```
}
```

```
// Find Least Recently Used page
```



```
int check()
{
int min = status[0];
int pos = 0;
for(int i = 1; i < 3; i++)
{
if(status[i] < min)
{
min = status[i];
pos = i;
}
}
return pos;
}

int main()
{
int i, j;
int hit = 0, fault = 0;
// Initialize first 3 pages
for(i = 0; i < 3; i++)
{
sol[i] = input[i];
```



```
status[i] = i;

}

printf("Number of frames is 3\n");

// Print initial frames

for(i = 0; i < 3; i++)

printf("%d", sol[i]);

printf("\n");

// Process remaining pages

for(j = 3; j < 20; j++)

{

int pos = present(input[j]);

if(pos == -1) // PAGE FAULT

{

int replace = check();

sol[replace] = input[j];

status[replace] = j;

fault++;

}

else // PAGE HIT

{

hit++;

status[pos] = j;
```



```
}  
  
// Print current frame state  
  
for(i = 0; i < 3; i++)  
  
printf("%d", sol[i]);  
  
printf("\n");  
  
}  
  
printf("\nNo. of hits = %d\n", hit);  
  
printf("No. of faults = %d\n", fault);  
  
return 0;  
  
}
```

Output:

```
apsit@apsit-HP-280-G3-MT:~/Shriya$ gcc -o xyz exp8.c  
apsit@apsit-HP-280-G3-MT:~/Shriya$ ./xyz  
Number of frames is 3  
701  
201  
201  
203  
203  
403  
402  
432  
032  
032  
032  
132  
132  
102  
102  
107  
107  
107
```

Conclusion: Thus we have studied that the primary purpose of a page replacement algorithm is to manage limited physical memory (RAM) efficiently by deciding which memory pages to swap out to secondary storage when new pages must be loaded. Also It minimizes page faults and reduces system latency, thereby improving overall performance by keeping actively used pages in memory.



Exercise 1

Program:

```
#include <stdio.h>
```

```
int input[20] = {7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};
```

```
int frame[3];
```

```
int time[3];
```

```
int present(int p)
```

```
{  
    for(int i = 0; i < 3; i++)  
        if(frame[i] == p)  
            return i;  
    return -1;  
}
```

```
int lru()
```

```
{  
    int min = time[0], pos = 0;  
    for(int i = 1; i < 3; i++)  
    {  
        if(time[i] < min)  
        {  
            min = time[i];  
            pos = i;  
        }  
    }  
    return pos;  
}
```

```
int main()
```

```
{  
    int hit = 0, fault = 0;  
  
    // Initialize frames  
    for(int i = 0; i < 3; i++)  
    {  
        frame[i] = -1;  
    }  
}
```



```
time[i] = -1;
}

for(int i = 0; i < 20; i++)
{
    int pos = present(input[i]);

    if(pos != -1)
    {
        hit++;
        time[pos] = i;
    }
    else
    {
        int replace = -1;

        for(int j = 0; j < 3; j++)
        {
            if(frame[j] == -1)
            {
                replace = j;
                break;
            }
        }

        if(replace == -1)
            replace = lru();

        frame[replace] = input[i];
        time[replace] = i;
        fault++;
    }
}

printf("Hits = %d\n", hit);
printf("Faults = %d\n", fault);

printf("Hit Ratio = %.2f\n", (float)hit/20);
printf("Miss Ratio = %.2f\n", (float)fault/20);
```



```
    return 0;  
}
```

Output:

```
apsit@apsit-HP-280-G3-MT:~/Shriya$ gcc -o xyz exp8a.c  
apsit@apsit-HP-280-G3-MT:~/Shriya$ ./xyz  
Hits = 8  
Faults = 12  
Hit Ratio = 0.40  
Miss Ratio = 0.60  
apsit@apsit-HP-280-G3-MT:~/Shriya$ |
```

Exercise 2

Program:

```
#include <stdio.h>
```

```
int ref[12] = {4,3,2,1,4,3,5,4,3,2,1,5};
```

```
int n = 12;
```

```
int frames = 3;
```

```
// FIFO
```

```
int fifo()
```

```
{
```

```
    int f[3] = {-1,-1,-1};
```

```
    int front = 0, faults = 0;
```

```
    for(int i = 0; i < n; i++)
```

```
    {
```

```
        int found = 0;
```

```
        for(int j = 0; j < frames; j++)
```

```
        {
```

```
            if(f[j] == ref[i])
```

```
            {
```

```
                found = 1;
```

```
                break;
```

```
            }
```

```
        }
```



```
if(!found)
{
    f[front] = ref[i];
    front = (front + 1) % frames;
    faults++;
}
}
return faults;
}
```

```
// LRU
int lru()
{
    int f[3] = {-1,-1,-1};
    int time[3] = {0};
    int faults = 0;

    for(int i = 0; i < n; i++)
    {
        int found = -1;

        for(int j = 0; j < frames; j++)
        {
            if(f[j] == ref[i])
                found = j;
        }

        if(found != -1)
        {
            time[found] = i;
        }
        else
        {
            int pos = 0;

            for(int j = 1; j < frames; j++)
                if(time[j] < time[pos])
                    pos = j;
        }
    }
}
```




```
f[pos] = ref[i];
time[pos] = i;
faults++;
}
}
return faults;
}

// Optimal
int optimal()
{
    int f[3] = {-1,-1,-1};
    int faults = 0;

    for(int i = 0; i < n; i++)
    {
        int found = 0;

        for(int j = 0; j < frames; j++)
        {
            if(f[j] == ref[i])
            {
                found = 1;
                break;
            }
        }

        if(!found)
        {
            int pos = -1, farthest = i;

            for(int j = 0; j < frames; j++)
            {
                int k;
                for(k = i + 1; k < n; k++)
                {
                    if(f[j] == ref[k])
                        break;
                }
            }
        }
    }
}
```



```
        if(k == n)
        {
            pos = j;
            break;
        }

        if(k > farthest)
        {
            farthest = k;
            pos = j;
        }
    }

    if(pos == -1)
        pos = 0;

    f[pos] = ref[i];
    faults++;
}
}
return faults;
}

int main()
{
    printf("FIFO Faults = %d\n", fifo());
    printf("LRU Faults = %d\n", lru());
    printf("Optimal Faults = %d\n", optimal());

    return 0;
}
```

Output:

```
apsit@apsit-HP-280-G3-MT:~/Shriya$ gcc -o xyz exp8b.c
apsit@apsit-HP-280-G3-MT:~/Shriya$ ./xyz
FIFO Faults = 9
LRU Faults = 10
Optimal Faults = 7
apsit@apsit-HP-280-G3-MT:~/Shriya$ |
```